Fachhochschule Nordwestschweiz
Hochschule für Technik

Multi-Layer Perceptrons (MLP)

Deep Learning

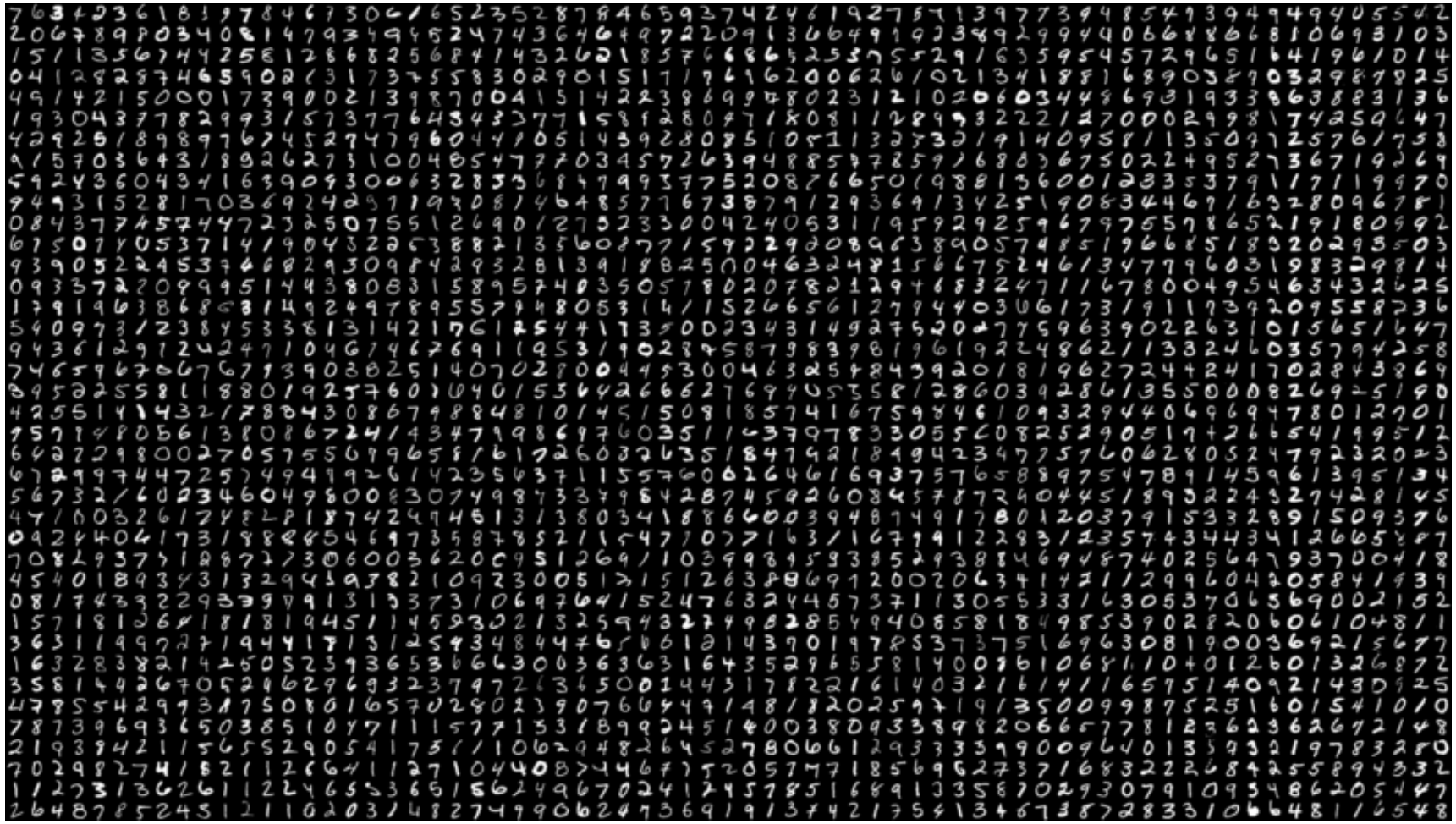
FS 2026

Martin Melchior

Agenda

- Loading Data, MNIST
- Multi-Layer Perceptron (MLP)
- Stochastic Gradient Descent (SGD)
- Convolutional Neural Network (CNN)
- Outlook: Training *Deep* Neural Networks

Loading Data, MNIST



Data Loading in PyTorch

Files, RAM, DB, …

class inherited from torch.utils.data.Dataset

- indexed (random) access to the samples e.g. (x, y)-pairs
- configure pre-processing steps (i.e. normalisation), etc.

see blog for custom datasets

torch.utils.data.DataLoader

- iterator over batches of samples: batch of x's, batch of y's
- shuffling of samples when iterator is reset
- parallelised access
- memory pinning
- etc.

see blog article for some explanation and additional recommendations

MNIST Dataset with PyTorch

Use datasets.MNIST to load 60’000 images for training and 10’000 for testing

```
import torch
from torchvision import datasets
from torchvision.transforms import ToTensor
import matplotlib.pyplot as plt
```

```
training_data = datasets.MNIST(
    root="data",
    train=True,
    download=True,
    transform=ToTensor()
)
```

for data downloaded to a subfolder of your working directory.

`transform`
for preprocessing pipeline
`ToTensor`
Transforms PIL images or numpy-arrays to tensor. Scaling!

```
print(training_data)
```

```
Dataset MNIST
    Number of datapoints: 60000
    Root location: data
    Split: Train
    StandardTransform
Transform: ToTensor()
```

see torch.utils.data for further details on datasets and loading data

Loading MNIST with PyTorch

Elements are (x,y)-tuples with

x : a pytorch tensor with the 2d grayscale pixel values

y : an int with the label

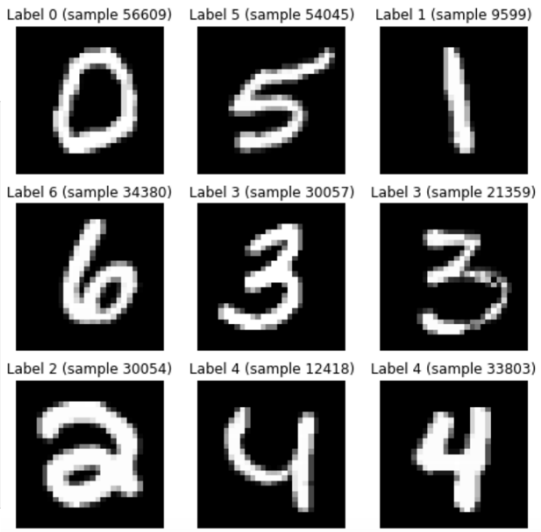
6th sample

```
print(len(training_data))
x, y = training_data[5]
print(type(x), x.shape, x.dtype)
print(type(y), y)
```

tensor with shape 1x28x28 with the first dimension corresponding to the color channel

```
60000
<class 'torch.Tensor'> torch.Size([1, 28, 28]) torch.float32
<class 'int'> 2
```

```
figure = plt.figure(figsize=(8, 8))
cols, rows = 3, 3
for i in range(1, cols * rows + 1):
    sample_idx = torch.randint(len(training_data), size=(1,)).item()
    img, label = training_data[sample_idx]
    figure.add_subplot(rows, cols, i)
    plt.title("Label %i (sample %i)"%(label,sample_idx))
    plt.axis("off")
    plt.imshow(img.squeeze(), cmap="gray")
plt.show()
```



DataLoader Class in PyTorch

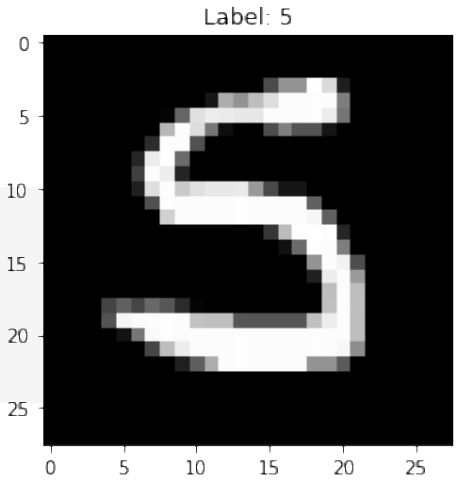
```
from torch.utils.data import DataLoader
train_dataloader = DataLoader(training_data, batch_size=64, shuffle=True)
```

```
data_train, labels_train = next(iter(train_dataloader))
print(f"Feature batch shape: {data_train.shape}")
print(f"Labels batch shape: {labels_train.shape}")
```

```
Feature batch shape: torch.Size([64, 1, 28, 28])
Labels batch shape: torch.Size([64])
```

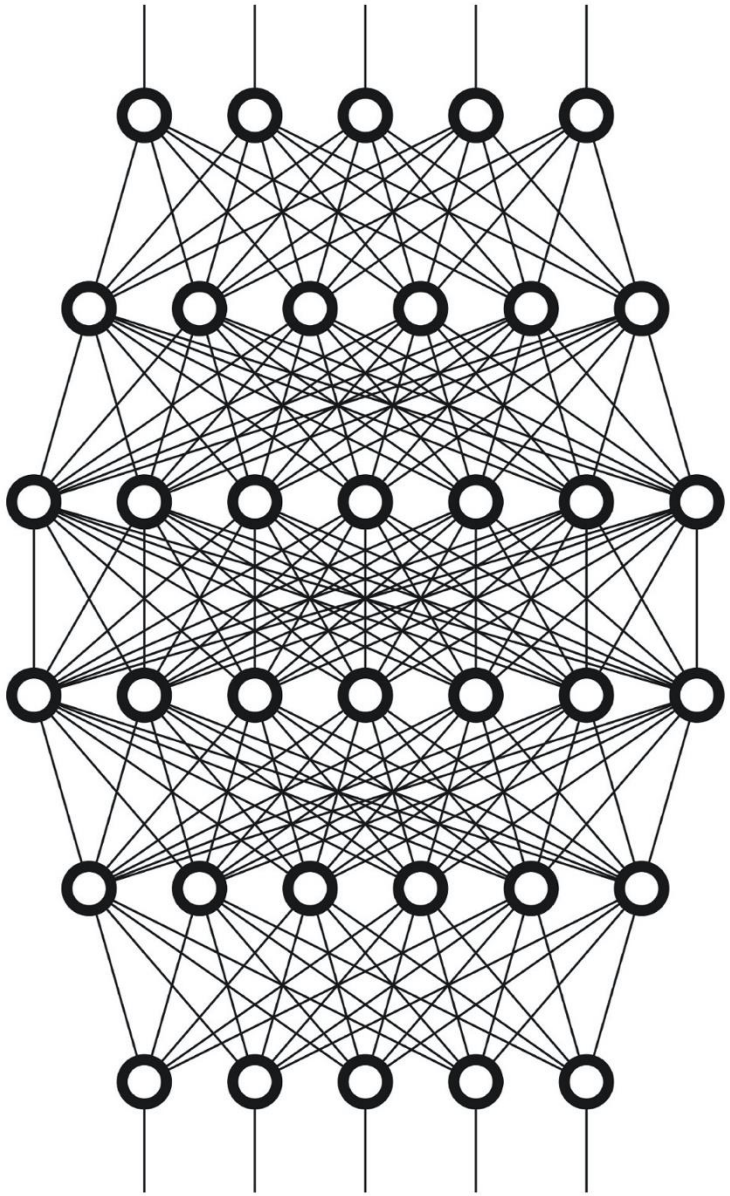
tensor with shape (64x1x28x28) with the first dimension corresponding to the sample index in the batch

```
img = data_train[0].squeeze()
label = labels_train[0]
plt.imshow(img, cmap="gray")
plt.title(f"Label: {label}")
plt.show()
```

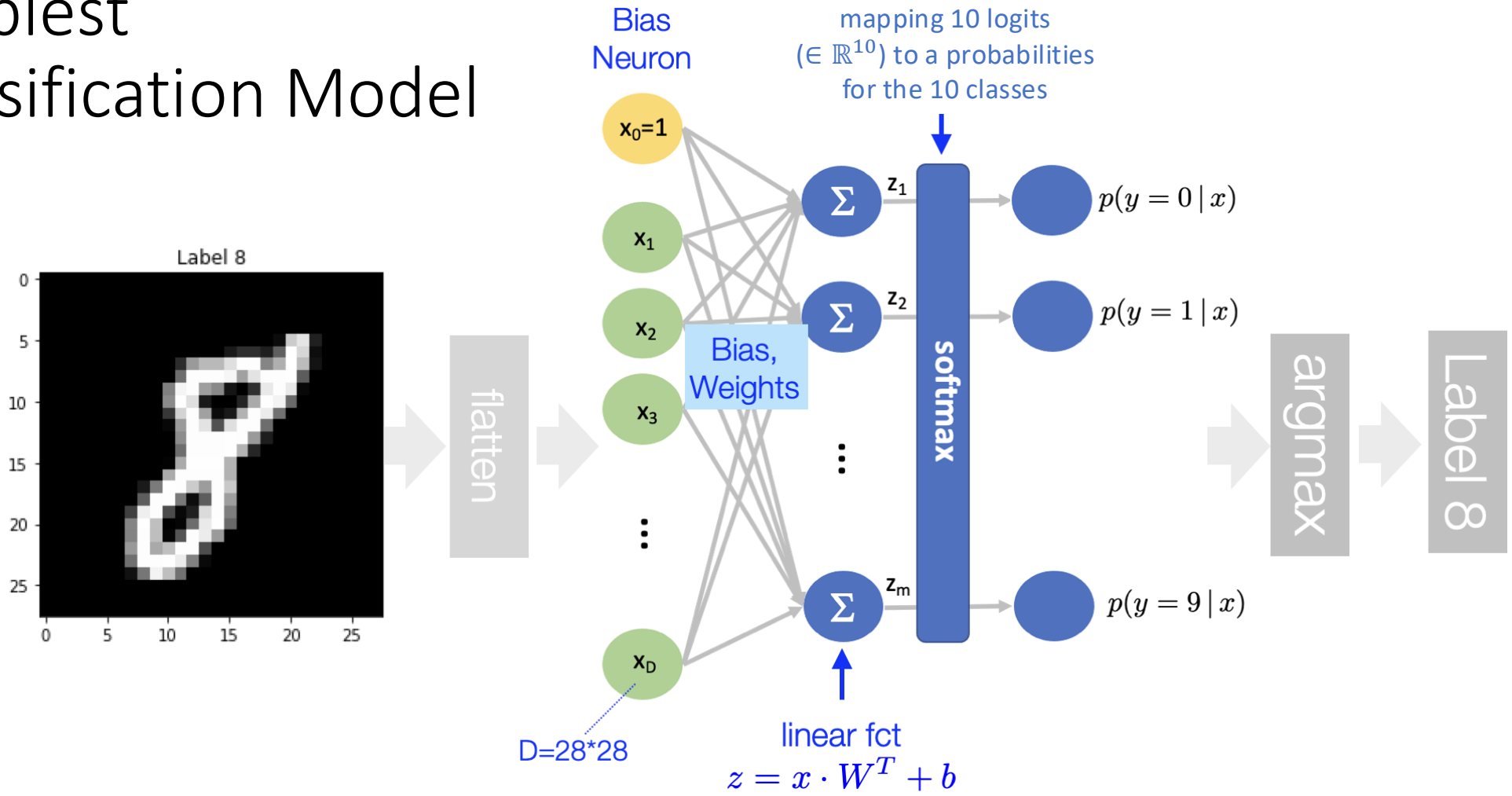


Fachhochschule Nordwestschweiz
Hochschule für Technik

Multi-Layer Perceptron



Simplest Classification Model



Multinomial Logistic Regression

Linear function: $z(x; W, b) = x \cdot W^\intercal + b$ "logits"

input	x	:	$(1, 28 * 28)$	flattened
logit	z	:	$(1, 10)$	
weights	W	:	$(10, 28 * 28)$	parameters θ
bias	b	:	$(1, 10)$	

Softmax function: $s_k(z) = \dfrac{e^{z_k}}{\sum_{j=1}^{10} e^{z_j}}, \quad k = 1, \ldots, 10$

"squeezing"

$$s_k(z) > 0\,, \sum_{k=1}^{10} s_k(z) = 1$$

probability interpretation !

Model function: $f_k(x; W, b) = s_k(z(x; W, b))$

(not linear in parameters!)

Model prediction: $\hat{y} = \underset{k}{\operatorname{argmax}} f_k(x; W, b)$

Gradient for Multinomial Logistic Regression

$$
\begin{aligned}
L_{\mathrm{CE}}(u, s(z(x, W, b))) &= -\sum_k u_k \log(s_k(z)) \\
&= -\sum_k u_k \log\left(\frac{e^{z_k}}{\sum_j e^{z_j}}\right) \\
&= -\sum_k u_k z_k + \sum_k u_k \log\left(\sum_j e^{z_j}\right) \\
&= -\sum_k u_k z_k + \log\left(\sum_j e^{z_j}\right)
\end{aligned}
$$

s: softmax

z: logits - linear in W, b

u: one-hot label vector

$$\frac{\partial L_{\mathrm{CE}}}{\partial z_k} = -u_k + s_k \qquad \frac{\partial z_k}{\partial W_{ij}} = \delta_{ik} x_j \qquad \frac{\partial z_k}{\partial b_i} = \delta_{ik}$$

$$\boxed{\frac{\partial L_{\mathrm{CE}}}{\partial W_{ij}} = (s_i - u_i) x_j \,, \qquad \frac{\partial L_{\mathrm{CE}}}{\partial b_i} = (s_i - u_i)}$$

$$
\begin{aligned}
W^{(t+1)} &= W^{(t)} - \alpha \frac{1}{N} \sum_i (\hat{u}^{(i)} - u^{(i)}) \left(x^{(i)}\right)^{\mathsf{T}} \\
b^{(t+1)} &= b^{(t)} - \alpha \frac{1}{N} \sum_i (\hat{u}^{(i)} - u^{(i)})
\end{aligned}
$$

Implementation in pytorch

```python
class LogisticRegression(torch.nn.Module):

    def __init__(self):
        super(LogisticRegression, self).__init__()
        self.flatten = torch.nn.Flatten()
        self.linear  = torch.nn.Linear(28*28,10)

    def forward(self, x):
        z = self.linear(self.flatten(x))
        return z
```

Every pytorch model is a subclass of torch.nn.Module

No softmax needed
– when using torch.nn.CrossEntropyLoss

Define the execution order:
First flatten, the apply the linear layer

```python
model = LogisticRegression()
print(model)
```

```
LogisticRegression(
  (flatten): Flatten(start_dim=1, end_dim=-1)
  (linear): Linear(in_features=784, out_features=10, bias=True)
)
```

```python
summary(model, (1,28,28))
```

```
----------------------------------------------------------------
        Layer (type)               Output Shape         Param #
================================================================
           Flatten-1                  [-1, 784]               0
            Linear-2                   [-1, 10]           7,850
================================================================
Total params: 7,850
Trainable params: 7,850
Non-trainable params: 0
----------------------------------------------------------------
Input size (MB): 0.00
Forward/backward pass size (MB
Params size (MB): 0.03
Estimated Total Size (MB): 0.04
----------------------------------------------------------------
```

To use this model summary you need torchsummary (install with «`pip install torchsummary`»)

Hands-On

mlp_classification.ipynb

Stochastic Gradient Descent: Training with Mini-Batches

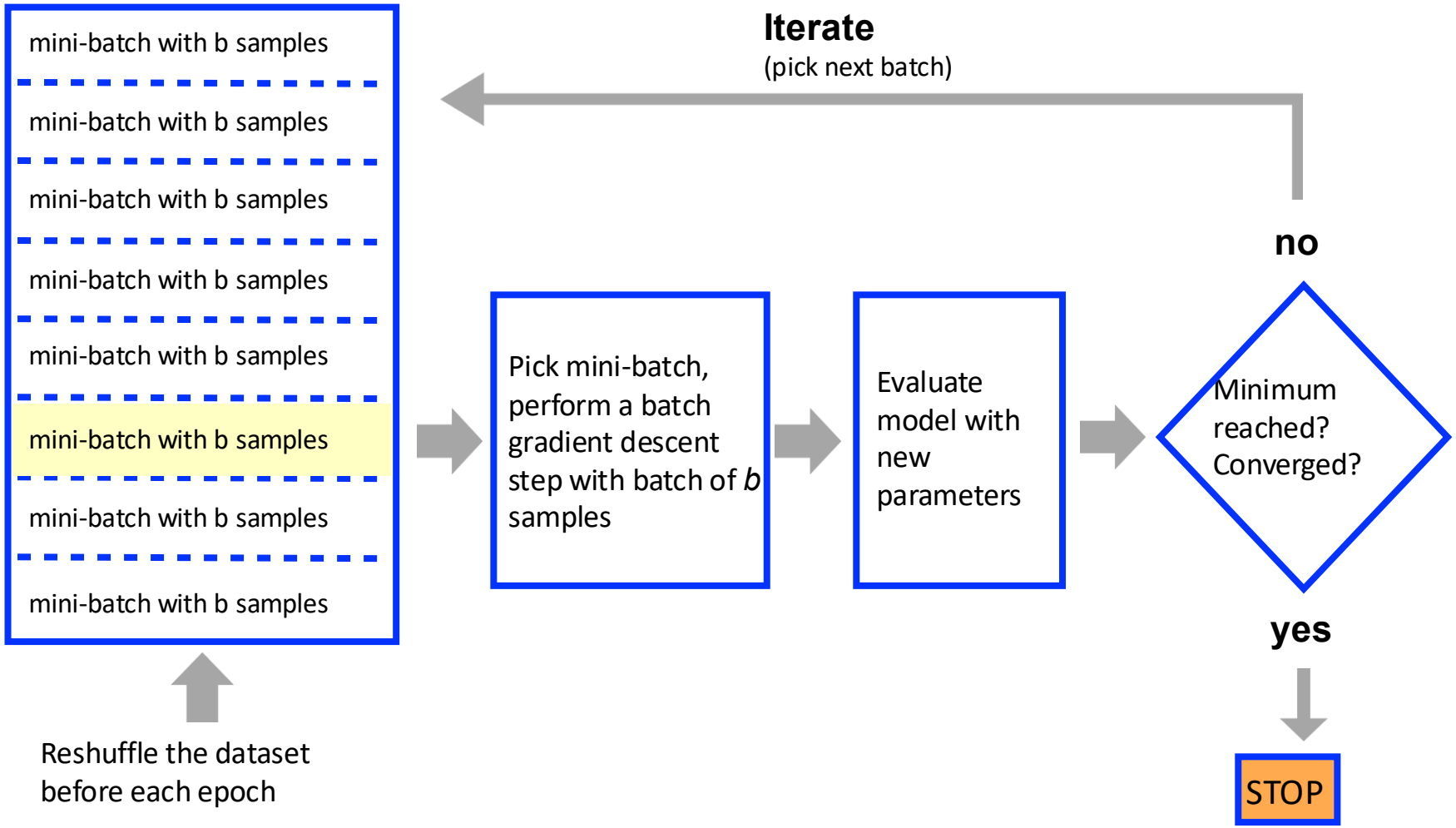
- Steepest descent direction given by the negative gradient is not optimal - no need to estimate it by using all the (training) samples.
- Use a subset of samples i.e. a batch ("mini-batch"):

$$\nabla C(\theta) \approx \frac{1}{N_b} \sum_{i:\text{batch}} \nabla_\theta L(y^{(i)}, f(x^{(i)}; \theta)$$

Proxy for the gradient, but its negative not necessary worse as descent direction!

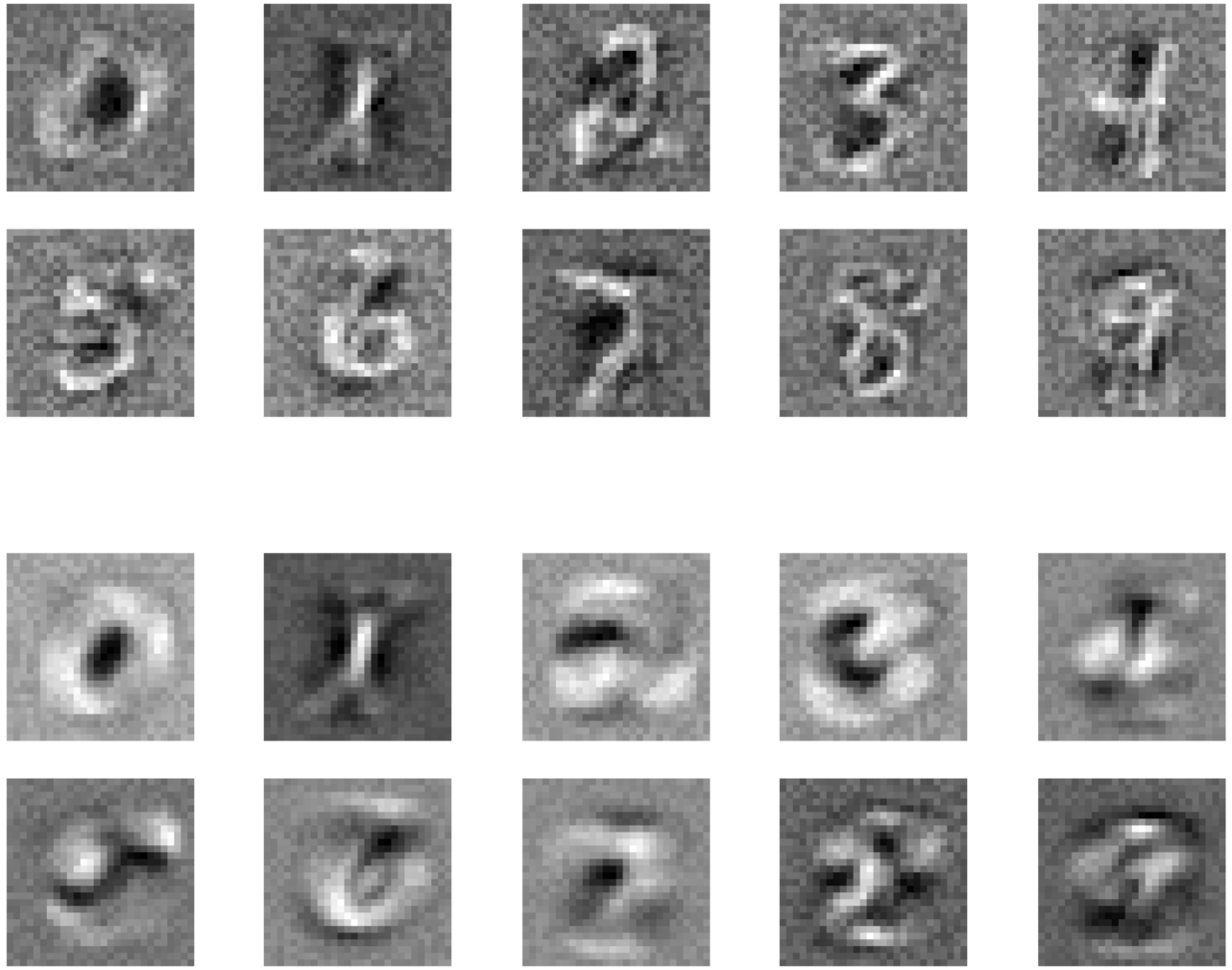
- Update parameters with gradients computed per batch -> more frequent updates -> performance gain!
- If updating the parameters only after one epoch we refer to it as "Batch Gradient Descent", otherwise as "Mini-Batch Gradient Descent" or "Stochastic Gradient Descent" (SGD).

Mini-Batch Gradient Descent Principle



Iterator and reshuffling provided by PyTorch's DataLoader

Hands-On



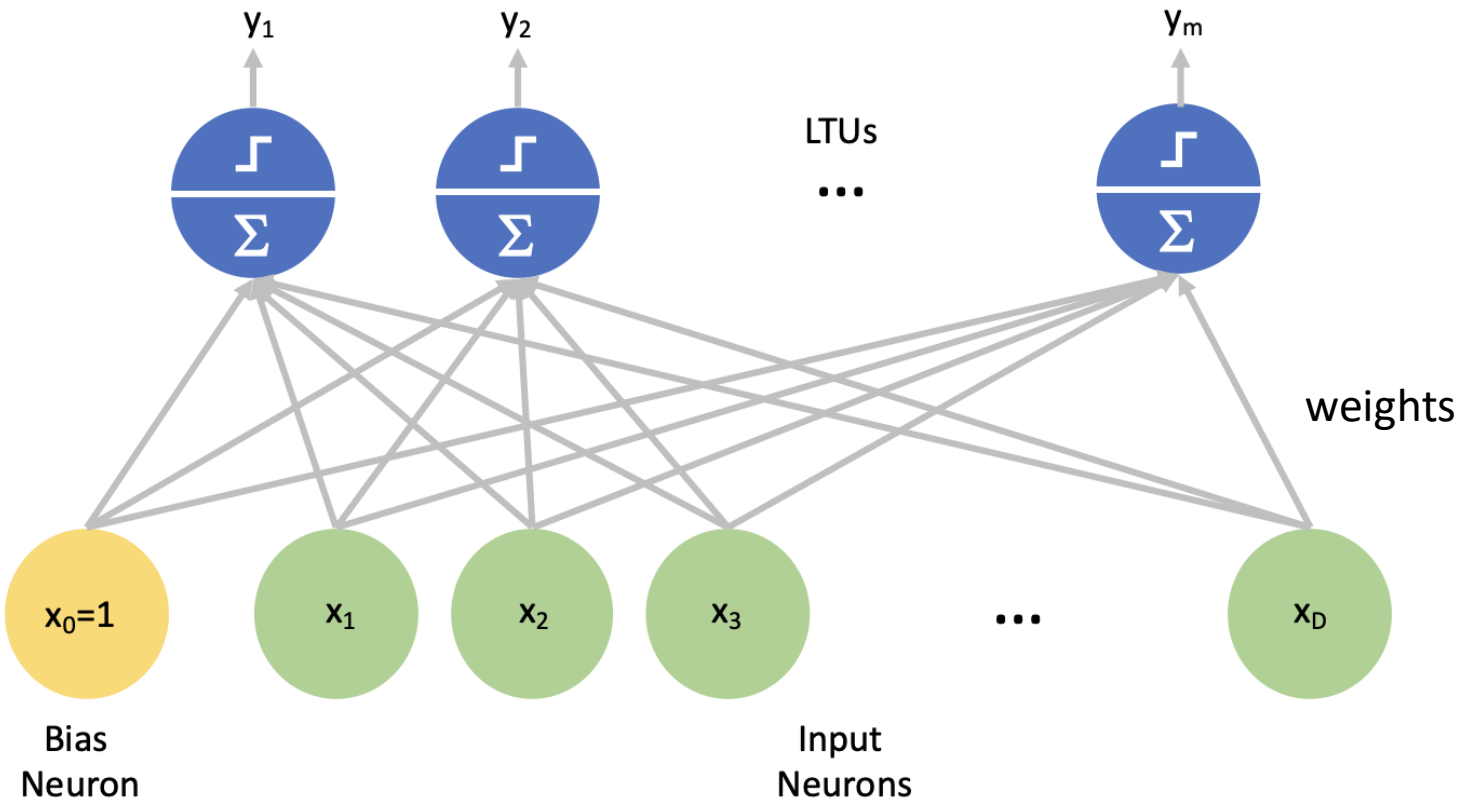
Perceptron Layer

Activation Function

Linear-Affine Transformation

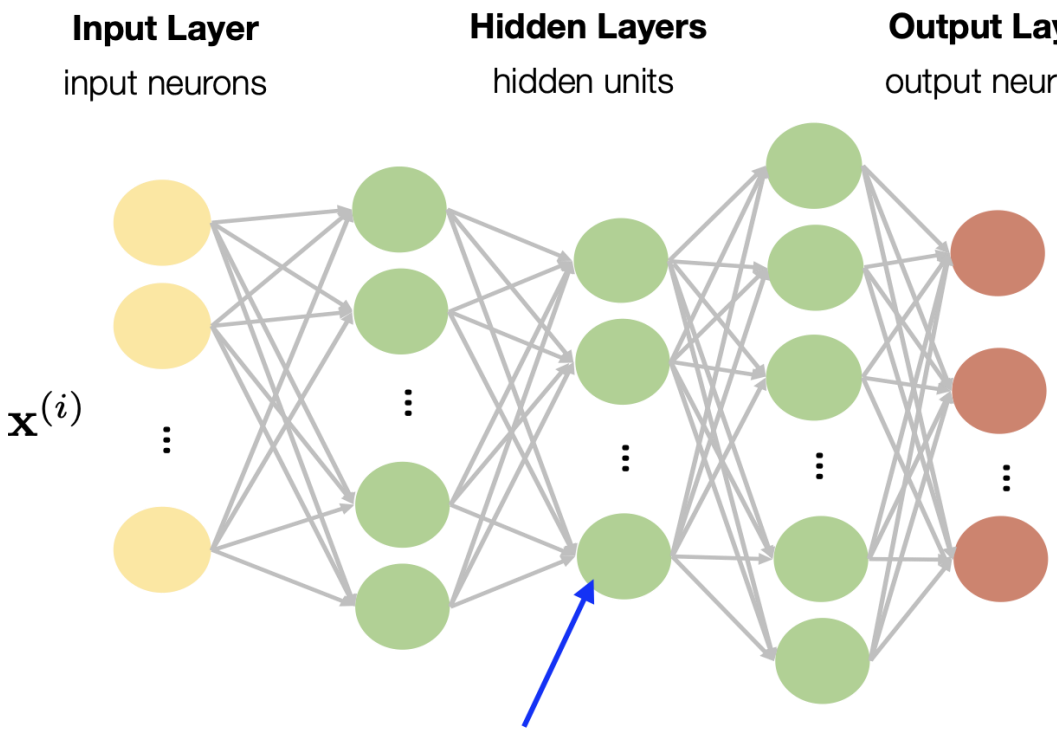
Input

or previous layer's output



Weights and bias to be trained

Stacking Perceptrons: Combining Features



$$\hat{u}^{(i)} = f_\alpha(x^{(i)})$$

Each unit ("neuron") computes a weighted sum of the output from the previous layer and applies an activation function.

Output is a non-linear mapping of the input array (tensor), parametrised by all the weights and biases of the different layers.

Hands-On